

Laporan Praktikum — Minggu 1 (JS-DVO-01)

Nama / NIM : Gandhi / 4332511050

Kelas : RKS305 — Rekayasa Keamanan Siber · Pasangan : Brian Polido Gultom (NIM 4332511660)

Repo : <https://github.com/GJRRR/praktikum-devops-m01>

Format akhir: PDF mengikuti template laporan praktikum Polibatam, nama berkas M01_NIM_NamaLengkap.pdf. Isi bagian ber-[DATA] dengan angka hasil pengukuran kami sendiri (JOB 2–4).

1. Analisis akar masalah

Pertanyaan: Mengapa akar masalahnya bukan kelalaian individu Developer, melainkan cacat desain sistem kerja? Sebutkan dua mekanisme sistemik.

Jawaban:

Kegagalan terjadi **1 kali dari 3 percobaan** (attempt 1: `ModuleNotFoundError`; attempt 2: `pip3 install flask` → sudah terpasang secara kebetulan; attempt 3: berhasil). Informasi yang hilang **selalu sama** di setiap kegagalan: `requirements.txt` **tidak ikut dalam paket serah-terima**. Jika penyebabnya kelalaian individu, pola kegagalannya akan acak. Kenyataannya pola ini berulang dan reproducible — itu ciri cacat **sistem kerja**, bukan kelalaian sesekali.

Bukti pada JOB 2: `HANDOVER.md` tidak memuat **daftar dependensi**; `requirements.txt` tidak disertakan dalam paket serah-terima/; mesin Ops tidak punya **cara reproducibility** (tidak ada cara verifikasi environment sama dengan Dev). Developer tidak pernah diminta oleh sistem apa pun untuk membuktikan aplikasinya bisa jalan di luar laptopnya sendiri.

- **Mekanisme sistemik 1 — Dependency & environment as code:** `requirements.txt` terkunci di repo + virtual environment + skrip `setup.sh`. Lingkungan didefinisikan oleh **berkas di repository**, bukan ingatan manusia. Setiap mesin yang menjalankan `./setup.sh` akan mendapat environment identik tanpa perlu bertanya ke siapa pun.
- **Mekanisme sistemik 2 — Automated verification gate:** smoke test `GET /health` + exit code 0/1. Artefak yang tidak lulus verifikasi otomatis **tidak pernah sampai ke tangan Ops** — galat tertangkap di pipeline sebelum menjadi insiden.
- **Mekanisme sistemik 3 (opsional) — Definition of Done yang terukur:** "Selesai" = `./setup.sh` lulus di **mesin bersih** (bukan di mesin Dev saja). Selama Dev hanya mengandalkan "jalan di laptop saya", definisi selesai selalu bias.

2. Evaluasi klaim otomasi (studi kasus Jenkins + Kubernetes + approval 3 hari)

Pertanyaan: Evaluasi dengan CALMS dan DORA, tentukan pilar mana yang gagal.

Jawaban:

Pilar CALMS	Kondisi perusahaan	Terpenuhi?	Alasan singkat
Culture	Dev & Ops masih terpisah, keputusan lewat eskalasi manajer	X	Shared responsibility belum ada; "bukan urusan saya" masih berlaku
Automation	Perkakas ada (Jenkins + K8s), tetapi langkah deploy belum otomatis	X	Tools terpasang tapi workflow manual; perkakas ≠ otomasi

Lean	Approval 3 hari = antrean murni	X	Waktu approval adalah wait time, bukan nilai tambah — murni pemborosan dari sudut pandang pelanggan
Measurement	Tidak ada metrik DORA yang dipakai	X	Tidak ada pengukuran Lead Time, MTTR, atau Change Failure Rate
Sharing	Pengetahuan berpusat pada manajer Ops	X	Bus factor = 1; jika manajer sakit/lupa, sistem macet

Kesimpulan inti: membeli perkakas ≠ DevOps. Pilar **Automation** gagal karena alat tidak menggantikan keputusan manual (approval tetap manusia); **Lean** gagal karena approval 3 hari adalah `_wait time_` murni; **Culture** gagal karena tanggung jawab tidak dibagi lintas fungsi.

Dampak ke DORA:

- **Lead Time for Changes** memburuk (+3 hari approval).
- **Deployment Frequency** turun (insentif menggabungkan banyak perubahan jadi satu rilis besar → batch size naik).
- **Change Failure Rate** ikut naik karena batch besar sulit di-review, sulit di-rollback.
- **MTTR** sulit diukur (tidak ada telemetry otomatis).

Satu kalimat penutup: tanpa mengubah cara kerja, Jenkins hanya menjadi **mesin yang lebih mahal dari email**.

3. Kuantifikasi dampak

Pertanyaan: Dengan Lead Time kami, hitung penghematan untuk 20 deployment/bulan, lalu sebut 2 manfaat non-waktu.

Lead Time manual (JOB 2) = 29 menit Lead Time otomatis (JOB 4) = 2 menit (1 perintah `./setup.sh` + verifikasi) Penghematan per deployment = 27 menit Penghematan 20 deployment/bulan = 540 menit = 9 jam/bulan

Dua manfaat yang tidak terukur oleh metrik waktu:

1. **Repeatability (kepastian hasil):** Jalankan `./setup.sh` 3× berturut-turut → selalu SUKSES dengan output identik. Tidak bergantung pada siapa yang menjalankan, urutan langkah, atau kondisi awal mesin. Cognitive load Ops turun drastis karena tidak perlu menghafal prosedur manual.
2. **Auditabilitas & onboarding cepat:** Setiap langkah terekam di repo (`setup.sh`, `teardown.sh`, `requirements.txt`, `app.log`, commit history) dan di log aplikasi. Insiden bisa ditelusuri via `git log + cat app.log`. Anggota baru cukup menjalankan `./setup.sh` — tidak perlu briefing berhari-hari tentang "environment seperti apa yang dipakai".

4. Perbandingan model SDLC secara kontekstual (alat kesehatan, regulasi ketat)

Pertanyaan: Model apa yang paling sesuai? Praktik DevOps mana yang tetap bisa diadopsi?

- **Model yang dipilih: Waterfall / V-Model bertahap berbasis dokumen.** Regulator (BPFK / FDA setara) mensyaratkan spesifikasi lengkap, `_design history file_`, dan `_traceability_` sebelum implementasi. Continuous deployment otomatis ke produksi tidak mungkin tanpa persetujuan regulator.
- **Tetap dapat diadopsi (prinsip DevOps):**

- **Version control + code review wajib** — bahkan **lebih penting** di sini karena menjadi jejak audit regulator.
- **Otomasi build & pengujian (CI)** — justru **memperkuat** bukti verifikasi regulator: setiap commit otomatis lewat test suite, hasilnya terekam.
- **Infrastructure as Code** untuk lingkungan pengujian yang reproducible — menghindari "drift" antara environment validasi dan produksi.
- **Pemantauan & logging pasca-rilis** — alat pacu jantung yang gagal detected cepat bisa menyelamatkan nyawa.
- **Tidak dapat diadopsi apa adanya:**
- **Continuous Deployment otomatis ke produksi** — wajib ada `_human gate_` + persetujuan regulator (mirip approval 3 hari pada skenario modul).
- **Canary release / eksperimen langsung pada pasien** — tidak etis dan tidak aman.
- **Perubahan tanpa jejak dokumen** — regulator menolak.
- **Catatan:** yang diadopsi adalah **prinsip otomasinya** (CI, IaC, monitoring), **bukan kebebasan merilis kapan saja**. Regulasi membatasi kecepatan, DevOps membatasi **variasi**.

5. Trade-off kecepatan dan stabilitas

Pertanyaan: Mengapa Deployment Frequency naik tidak otomatis menaikkan Change Failure Rate? Kapan tidak berlaku?

Jawaban:

Pada JOB 2, satu "rilis" berisi **seluruh aplikasi + konfigurasi tak tertulis** dalam satu paket → **1 kegagalan** (ModuleNotFoundError) yang baru ketahuan setelah Ops mencoba menjalankan. Pada JOB 4, perubahan kecil dan terverifikasi otomatis oleh `setup.sh` + health check → **0 kegagalan** dari 3x eksekusi.

Alasan:

- **Batch kecil** → diff mudah ditinjau (`_code review_` cepat, risiko rendah).
- **Blast radius sempit** → jika gagal, hanya perubahan kecil yang harus di-`_rollback_`.
- **Rollback cepat** → MTTR turun (estimasi < 5 menit untuk kembali ke versi sebelumnya via `git revert + ./setup.sh`).
- **Gerbang otomatis seragam** → setiap perubahan lulus uji yang **sama**, tidak ada "kebetulan berhasil seperti JOB 2".

Kondisi di mana asumsi TIDAK berlaku:

Deployment Frequency naik **tanpa** gerbang otomatis (tidak ada health check / unit test / rollback plan) atau tanpa observabilitas (tidak ada log, metric, alert) — maka kegagalan tidak terdeteksi, dan kecepatan justru memperbesar Change Failure Rate. **Syaratnya: otomasi verifikasi + observabilitas harus mengimbangi kecepatan rilis.**

6. Hasil Simulasi Board SDLC (JOB 5)

Dua board disimulasikan dengan 5 kartu yang mengalir lewat kolom:

Board	Alur kolom	Kolom dengan kartu tertahan terlama	Waktu total
-------	------------	-------------------------------------	-------------

Waterfall	Requirement → Design → Implementation → Testing → Deployment	Implementation (2:57)	08:28,41
DevOps Flow	Backlog → In Progress (WIP 2) → Review → Deployed → Monitored	Review (5:06)	10:21,48

Insight: Untuk satu task kecil, Waterfall (08:28) justru lebih cepat dari DevOps Flow (10:21) karena DevOps Flow menambah gerbang **Review** + batas **WIP** yang menjadi overhead. Keunggulan DevOps Flow bukan pada *cycle time* satu task, melainkan **throughput & parallelism**: dengan banyak task bersamaan, WIP limit mencegah antrean panjang dan Review berjalan paralel, sehingga total lead time rangkaian task jauh lebih pendek. Kesimpulan: pilih model sesuai konteks — Waterfall untuk task tunggal sederhana, DevOps Flow untuk aliran banyak perubahan berkesinambungan. Blameless postmortem dari simulasi ini ada di `POSTMORTEM.md`.

Refleksi pribadi (bahan diskusi, nilai +5)

- Hal yang paling mengubah cara pandang saya tentang Dev/Ops:

"Selama ini saya pikir DevOps = install Jenkins. Praktikum ini menunjukkan DevOps = **menghilangkan wall of confusion** lewat artefak yang mereproduksi dirinya sendiri (`requirements.txt` + `setup.sh`). Saya yang awalnya bingung 'kenapa Dev bilang sudah selesai tapi Ops bilang belum mulai' sekarang paham: sistemnya yang salah, bukan orangnya."

- Bagian sistem kerja kelompok kami yang masih manual setelah praktikum ini:

"Dokumentasi keputusan arsitektur (misal: kenapa pilih Flask, kenapa port 5000, kenapa WSL2) masih di kepala, belum jadi `_docs/architecture.md` di repo. Jika salah satu dari kami cuti 2 minggu, yang lain harus scroll commit history untuk menebak alasannya."

Lampiran (lihat di repo)

- Tabel pengukuran lengkap: `[dokumen/LEMBAR-KERJA.md](./LEMBAR-KERJA.md)`
- Value Stream Map + Flow Efficiency: `[dokumen/VSM.md](./VSM.md)`
- Blameless postmortem: `[. ./POSTMORTEM.md](../POSTMORTEM.md)`
- Skrip otomatisasi: `[. ./app-sentra/setup.sh](../app-sentra/setup.sh)`, `[. ./app-sentra/teardown.sh](../app-sentra/teardown.sh)`
- Bukti log environment: `00-environment-check.log`
- Tangkapan layar `setup.sh` sukses (Dev & Ops)
- Tangkapan layar `shellcheck setup.sh teardown.sh` bersih (Challenge)
- Tautan Projects Waterfall & DevOps Flow